

ED1 04: Filas

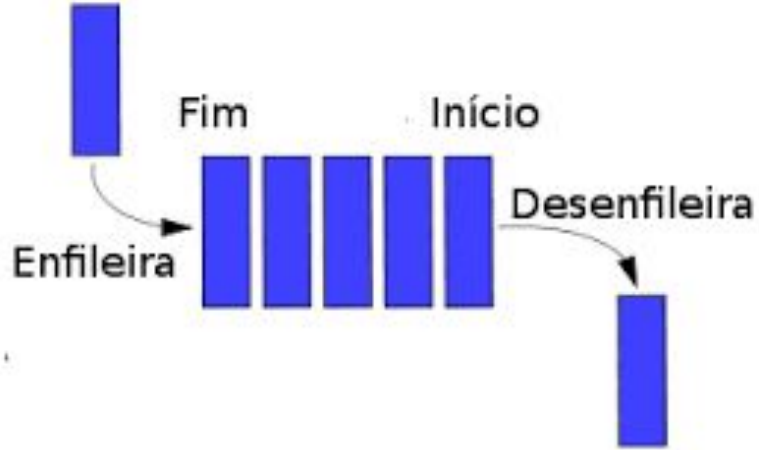
Fabio Irigon Pereira

Fila



Fila





Conceito de Fila (FIFO)

Uma fila (**queue**) é uma estrutura de dados do tipo **FIFO** (First In, First Out): o primeiro elemento a entrar é o primeiro a sair.

- enqueue (inserção no final)
- dequeue (remoção do início)
- front/peek (consulta do primeiro elemento)
- isEmpty / isFull

Tipos de filas

- Filas podem ser criadas estaticamente ou dinamicamente.
- Filas estáticas costumam usar buffers circulares.
- DEQUES (Double Ended QUEues) usam filas duplamente encadeadas e permitem adição e remoção em ambas extremidades.

Casos de uso

Pacotes de rede.

Telefone celular.

Escalonamento de processos.

Fila de impressão

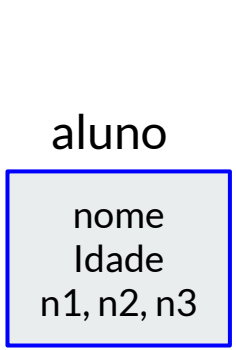
Operações em filas estáticas

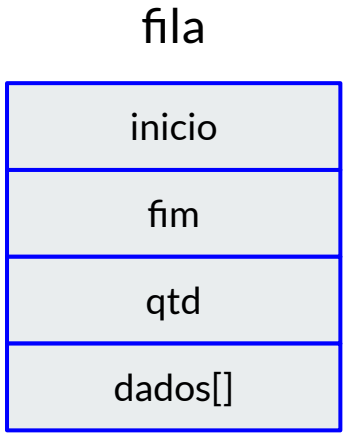
fonte: https://github.com/arbackes/Estrutura-de-Dados-em-C

Definição do elemento e funções

```
#define MAX 4

//Definição do tipo Fila
typedf struct Fila{
   int inicio, final, qtd;
   struct aluno dados[MAX];
}Fila;
```





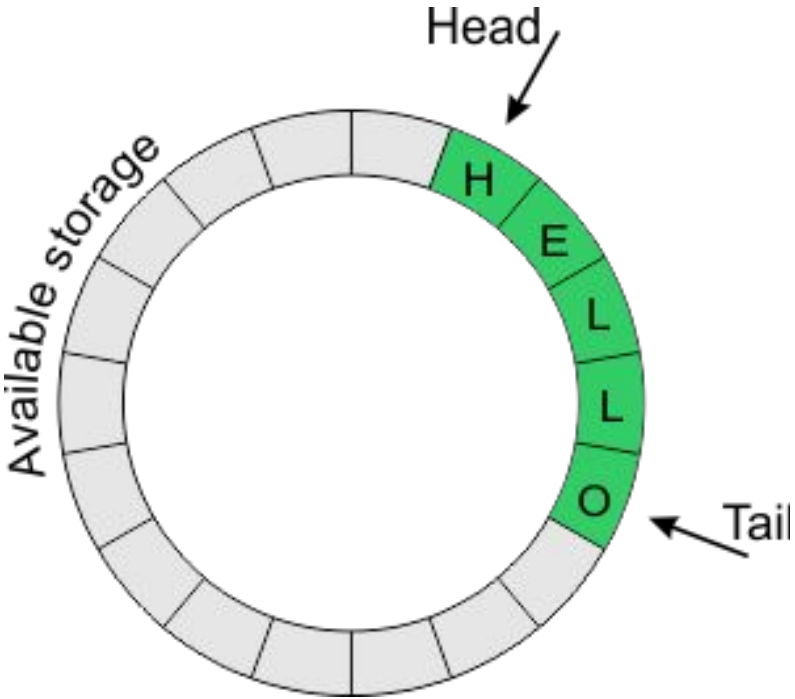
Criação e destruição de fila estática

```
Fila* cria_Fila(){
   Fila *fi;
   fi = (Fila*) malloc(sizeof(struct fila));
   if(fi != NULL){
        fi->inicio = 0;
        fi->final = 0;
        fi->qtd = 0;
   }
   return fi;
}

void libera_Fila(Fila* fi){
   free(fi);
}
```

Inserção Fila estática

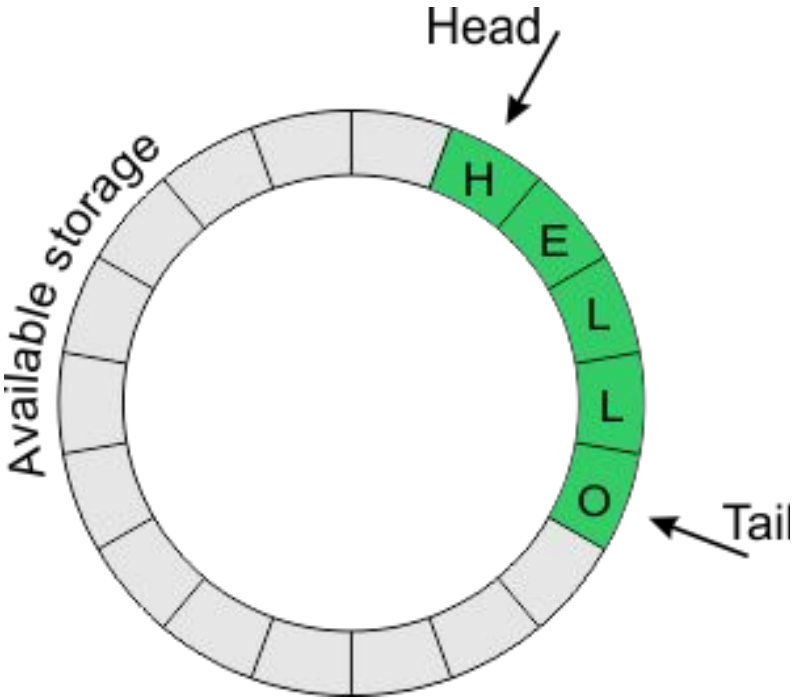
```
int insere_Fila(Fila* fi, struct aluno al){
    if(fi == NULL)
        return 0;
    if(Fila_cheia(fi))
        return 0;
    fi->dados[fi->final] = al;
    fi->final = (fi->final+1)%MAX;
    fi->qtd++;
    return 1;
}
```



Inserção Fila estática

```
int insere_Fila(Fila* fi, struct aluno al){
    if(fi == NULL)
        return 0;
    if(Fila_cheia(fi))
        return 0;
    fi->dados[fi->final] = al;
    fi->final = (fi->final+1)%MAX;
    fi->qtd++;
    return 1;
}
```

```
fi->final = fi->final+1;
if (fi->final == MAX){
    fi->final = 0
}
```



Consulta fila (início)

```
// Consulta topo (TOP)
int consulta_Fila(Fila* fi, struct aluno *al){
   if(fi == NULL || Fila_vazia(fi))
       return 0;
   *al = fi->dados[fi->inicio];
   return 1;
}
```

Remoção em Fila estática

```
// Remove um elemento do início da fila:
int remove_Fila(Fila* fi){
    if(fi == NULL || Fila_vazia(fi))
        return 0;
    fi->inicio = (fi->inicio+1)%MAX;
    fi->qtd--;
    return 1;
}
```

Exercícios:

Usando o exemplo com nome e idade:

1. Inserir 5 elementos e remover 1 da fila.
2. Construir a função front() (lê o primeiro mas não tira da fila)
3. Imprimir todos os elementos da fila.
4. Somar calcular a média de idade dos menores de 18 anos.
5. Inserir uma pessoa na penúltima posição da fila.

Operações em filas dinâmicas

fonte: https://github.com/arbackes/Estrutura-de-Dados-em-C

Definição dos tipos de dados

```
struct aluno{
   int matricula;
   char nome[30];
   float n1,n2,n3;
} aluno;
```

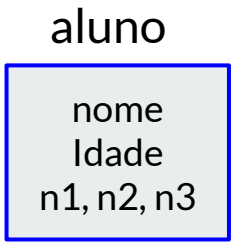
aluno

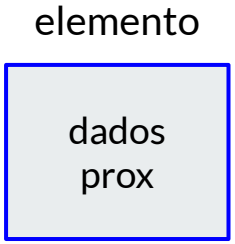
nome
Idade
n1, n2, n3

Definição dos tipos de dados

```
struct aluno{
   int matricula;
   char nome[30];
   float n1,n2,n3;
} aluno;

struct Elem{
   struct aluno dados;
   struct elemento *prox;
} Elem;
```





Definição dos tipos de dados

```
struct aluno{
   int matricula;
   char nome[30];
   float n1,n2,n3;
} aluno;

struct Elem{
   struct aluno dados;
   struct elemento *prox;
} Elem;

typedef struct Fila{
   struct elemento *inicio;
   struct elemento *final;
   int qtd;
} Fila;
```

aluno

nome
Idade
n1, n2, n3

elemento

dados
prox

fila

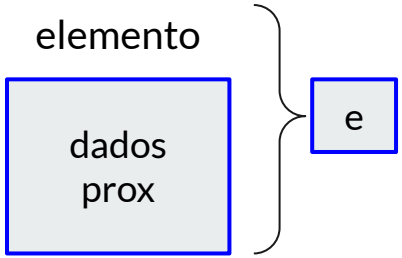
inicio
final
qtd

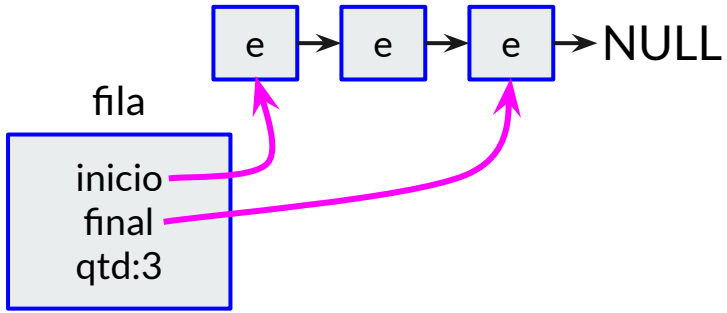
Definição dos tipos de dados

```
struct aluno{
   int matricula;
   char nome[30];
   float n1,n2,n3;
} aluno;

struct Elem{
   struct aluno dados;
   struct elemento *prox;
} Elem;

typedef struct Fila{
   struct elemento *inicio;
   struct elemento *final;
   int qtd;
} Fila;
```

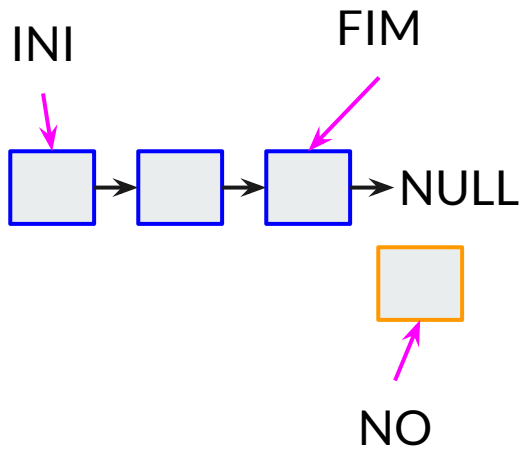




Inserção em fila dinâmica:

```
int insere_Fila(Fila* fi, struct aluno al){
   if(fi == NULL)
       return 0;
   Elem *no = (Elem*) malloc(sizeof(Elem));
   if(no == NULL)
       return 0;
   no->dados = al;
   no->prox = NULL;

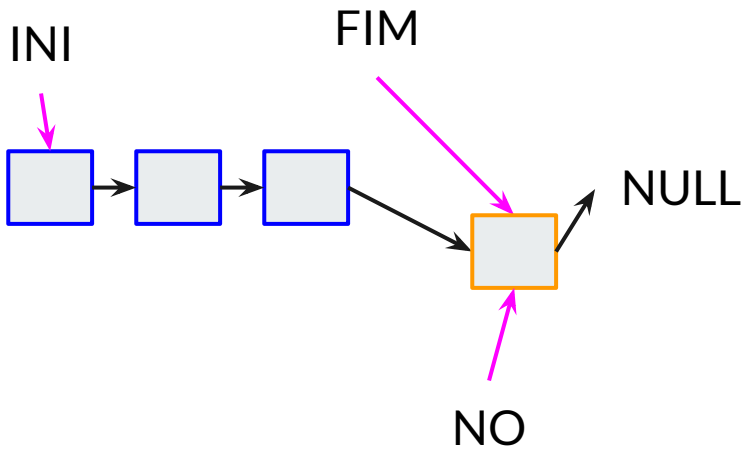
...
```



1) CRIA O NOVO NÓ

Inserção em fila dinâmica:

```
int insere_Fila(Fila* fi, struct aluno al){
   if(fi == NULL)
       return 0;
   Elem *no = (Elem*) malloc(sizeof(Elem));
   if(no == NULL)
       return 0;
   no->dados = al;
   no->prox = NULL;
   if(fi->final == NULL)//fila vazia
       fi->inicio = no;
   else
       fi->final->prox = no;
   fi->final = no;
   fi->qtd++;
   return 1;
}
```



2) AJUSTA OS PONTEIROS

Consulta início da fila:

```
int consulta_Fila(Fila* fi, struct aluno *al){
   if(fi == NULL)
       return 0;
   if(fi->inicio == NULL)//fila vazia
       return 0;
   *al = fi->inicio->dados;
   return 1;
}
```

Remover elemento da fila:

```
int remove_Fila(Fila* fi){
    if(fi == NULL)
        return 0;
    if(fi->inicio == NULL)//fila vazia
        return 0;
    Elem *no = fi->inicio;
```



```
    return 1;
}
```

Remover elemento da fila:

```
int remove_Fila(Fila* fi){
   if(fi == NULL)
       return 0;
   if(fi->inicio == NULL)//fila vazia
       return 0;
   Elem *no = fi->inicio;
   fi->inicio = fi->inicio->prox;
   if(fi->inicio == NULL)//fila ficou vazia
       fi->final = NULL;
   free(no);
   fi->qtd--;
   return 1;
}
```

Outras funções

Remover todos os elementos.

Imprimir elementos da fila.

Verificar se nomes estão em ordem alfabética.

Remover o alunos com média abaixo de 6.

Exercícios - agora com fila dinâmica:

Usando o exemplo:

1. Inserir 5 pessoas e remover uma da fila.
2. Imprimir todas as pessoas da fila.
3. Contar quantas pessoas da fila tem mais de 20 anos.
4. Buscar uma pessoa pelo nome.
5. Inserir uma pessoa entre a segunda e a terceira posição da fila.

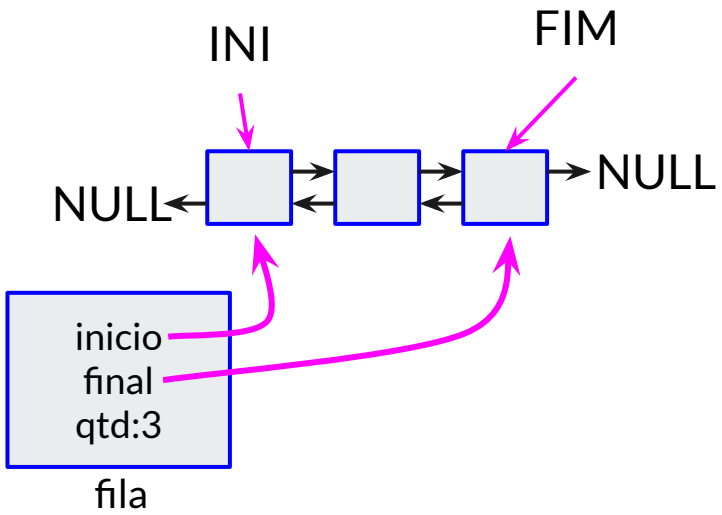
Operações em DEQUES

fonte: https://github.com/arbackes/Estrutura-de-Dados-em-C

Definição dos tipos de dados

```
typedef struct Elem{
   struct aluno dados;
   struct elemento *ant;
   struct Elem *prox;
}Elem;

typedef struct Deque{
   Elem *inicio;
   Elem *final;
   int qtd;
}Deque;
```



Insere no final

```
int insereFinal_Deque(Deque* dq, struct aluno al){
   if(dq == NULL)
       return 0;
   Elem *no = (Elem*) malloc(sizeof(Elem));
   if(no == NULL)
       return 0;
   no->dados = al;
   no->prox = NULL;
   if(dq->final == NULL){//Deque vazio
       no->ant = NULL;
       dq->inicio = no;
   }
   else{
       no->ant = dq->final;
       dq->final->prox = no;
   }
   dq->final = no;
   dq->qtd++;
   return 1;
}
```

Insere no início

```
int insereInicio_Deque(Deque* dq, struct aluno al){
   if(dq == NULL)
       return 0;
   Elem *no = (Elem*) malloc(sizeof(Elem));
   if(no == NULL)
       return 0;
   no->dados = al;
   no->prox = dq->inicio;
   no->ant = NULL;
   if(dq->inicio == NULL)
       dq->final = no;
   else//Deque não vazio: apontar para o anterior!
       dq->inicio->ant = no;
   dq->inicio = no;
   dq->qtd++;
   return 1;
}
```

Remove no início

```
int removeInicio_Deque(Deque* dq){
   if(dq == NULL)
       return 0;
   if(dq->inicio == NULL)//Deque vazio
       return 0;
   Elem *no = dq->inicio;
   dq->inicio = dq->inicio->prox;
   if(dq->inicio == NULL)//Deque ficou vazio
       dq->final = NULL;
   else
       dq->inicio->ant = NULL;
   free(no);
   dq->qtd--;
   return 1;
}
```

Exercícios

1. Imprimir todas as pessoas do deque.
2. Inserir uma pessoa em posição arbitrária do deque.
3. Remover uma pessoa pelo nome.

Combine as opções

1. **Histórico de abas do navegador:**
2. **Playlist de música (próxima/anterior):**
3. **Buffer de teclado (teclas digitadas):**
4. **Busca por CPF em base fixa de 1 milhão de nomes:**

A. (Pilha)
B. (Lista Duplamente Encadeada/Deque)
C. (Array ordenado/Busca Binária)
D. (Fila)

1 -
2 -
3 -
4 -